# Introduction to Artificial Intelligence

Lecture 6: Reasoning over time

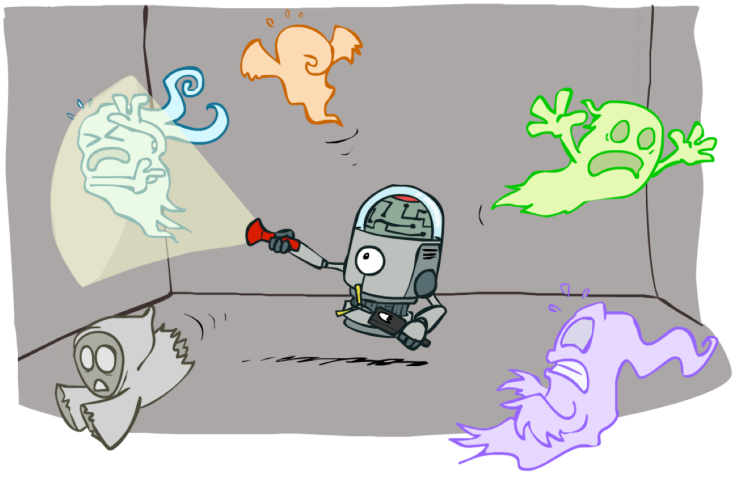
Prof. Gilles Louppe
g.louppe@uliege.be

LIÈGE université

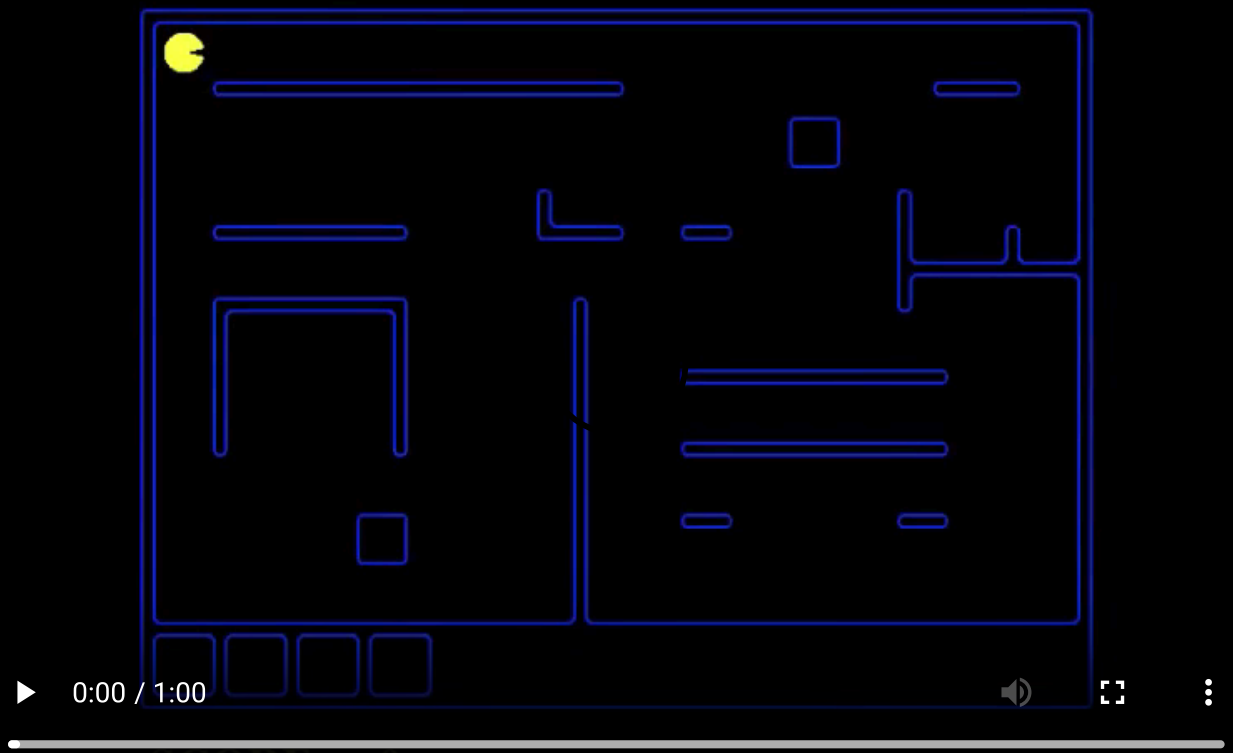
# Today

Maintain a **belief state** about the world, and update it as time passes and evidence is collected.

- Markov models
  - Markov processes
  - Inference tasks
  - Hidden Markov models
- Filters
  - Kalman filter
  - Particle filter



> Do not overlook this lecture!




**Pacman revenge**: How to make good use of the sonar readings?

# Markov models

## Stochastic, partially observable environments

In most environments, the agent neither sees the state nor controls how it evolves:

- the state moves at random, $s_{t+1} \sim P(s_{t+1} \mid s_t, a_t)$, following the **transition model**;
- the agent only perceives $e_t \sim P(e_t \mid s_t)$, following the **sensor model**.

A model-based agent therefore acts on a **belief state**, the distribution of the current state given everything perceived and done so far,

$$b_t = P(s_t \mid e_{1:t}, a_{1:t-1}),$$

which it updates at every step, $b_t = \mathrm{update}(b_{t-1}, a_{t-1}, e_t)$. This lecture derives $\mathrm{update}$.

## Modelling the passage of time

We consider the world as a **discrete** series of time slices, each of which contains sets of random variables:

- $\mathbf{X}_t$, the **unobservable** state variables at time $t$, which make up the state $s_t$;
- $\mathbf{E}_t$, the **observable** evidence variables at time $t$, which make up the percept $e_t$.

The world starts at $\mathbf{X}_0$, before any evidence, and the first observation is $\mathbf{E}_1$. The actions are taken as given and left out of the models.

We specify

- a prior $\mathbf{P}(\mathbf{X}_0)$ that defines our initial belief state over hidden state variables,
- a **transition model** $\mathbf{P}(\mathbf{X}_t \mid \mathbf{X}_{0:t-1})$ (for $t > 0$) that defines the probability distribution over the latest state variables, given the previous (unobserved) values,
- a **sensor model** $\mathbf{P}(\mathbf{E}_t \mid \mathbf{X}_{0:t}, \mathbf{E}_{0:t-1})$ (for $t > 0$) that defines the probability distribution over the latest evidence variables, given all previous (observed and unobserved) values.

# Markov processes

## Markov assumption

The current state of the world depends only on its immediate previous state(s), i.e., $\mathbf{X}_t$ depends on only a bounded subset of $\mathbf{X}_{0:t-1}$.

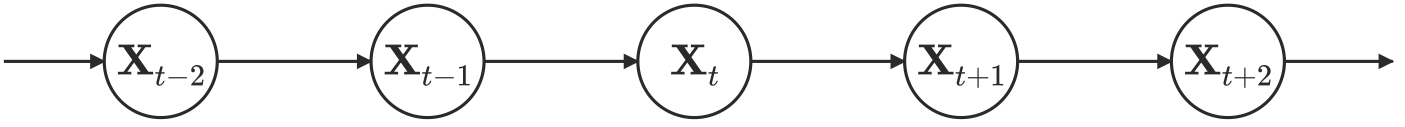
Random processes that satisfy this assumption are called **Markov processes** or **Markov chains**.

## First-order Markov processes

Markov processes such that

$$\mathbf{P}(\mathbf{X}_t \mid \mathbf{X}_{0:t-1}) = \mathbf{P}(\mathbf{X}_t \mid \mathbf{X}_{t-1})$$

i.e., $\mathbf{X}_t$ and $\mathbf{X}_{0:t-2}$ are conditionally independent given $\mathbf{X}_{t-1}$.



## Sensor Markov assumption

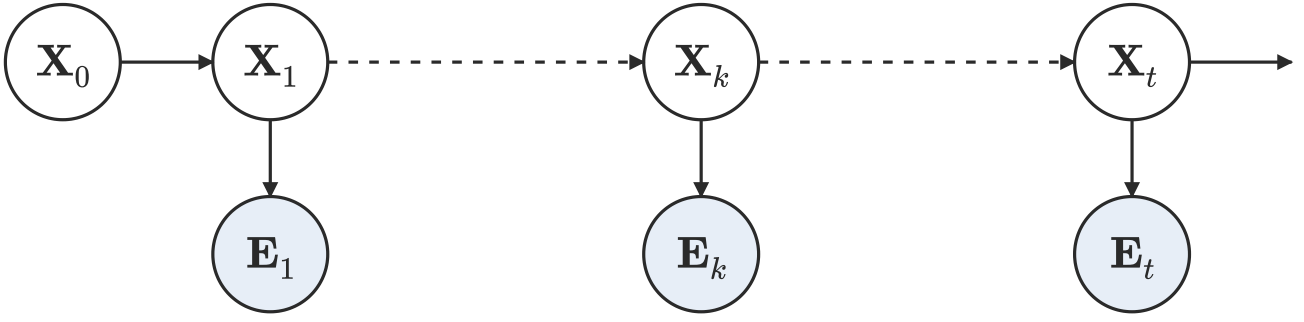
We make a (first-order) **sensor Markov assumption**

$$\mathbf{P}(\mathbf{E}_t \mid \mathbf{X}_{0:t}, \mathbf{E}_{0:t-1}) = \mathbf{P}(\mathbf{E}_t \mid \mathbf{X}_t).$$

## Stationarity assumption

The transition and the sensor models are the same for all $t$ (i.e., the laws of physics do not change with time).

# Joint distribution



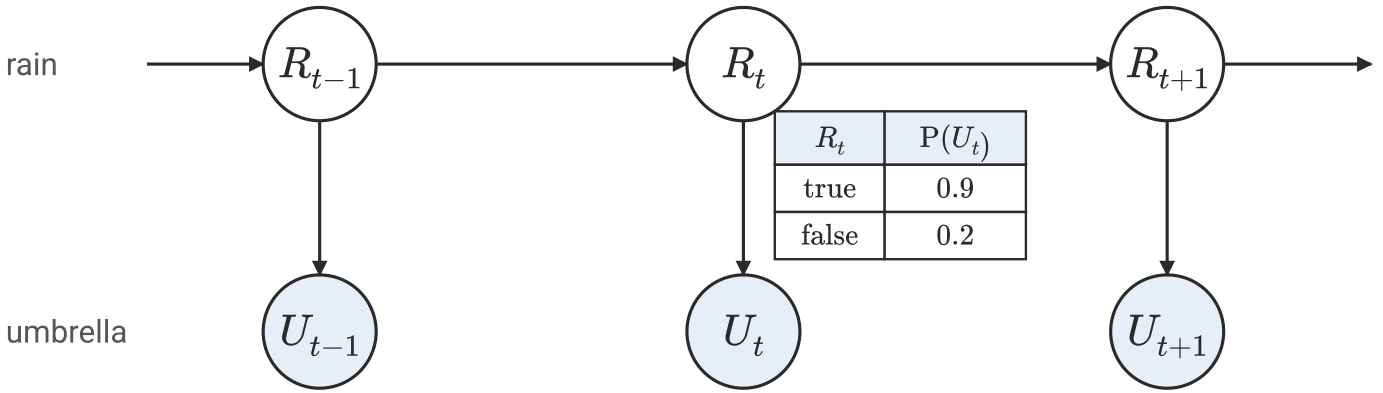

A Markov chain coupled with a sensor model can be represented as a **growable** Bayesian network, unrolled infinitely through time.

The joint distribution of all its variables up to $t$ is

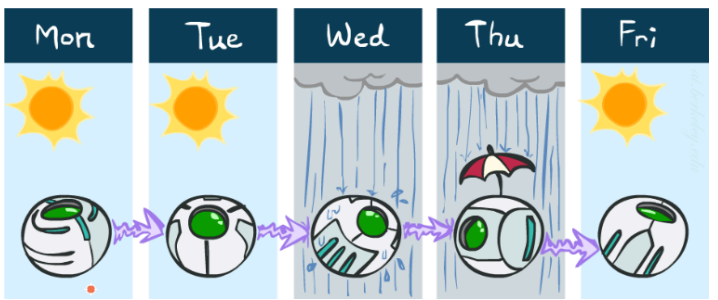
$$\mathbf{P}(\mathbf{X}_{0:t}, \mathbf{E}_{1:t}) = \mathbf{P}(\mathbf{X}_0) \prod_{i=1}^{t} \mathbf{P}(\mathbf{X}_i \mid \mathbf{X}_{i-1}) \mathbf{P}(\mathbf{E}_i \mid \mathbf{X}_i).$$

# Example: Will you take your umbrella today?

| $R_{t-1}$ | $\mathrm{P}(R_t)$ |
|---|---|
| true | 0.7 |
| false | 0.3 |




| $R_t$ | $\mathrm{P}(U_t)$ |
|---|---|
| true | 0.9 |
| false | 0.2 |

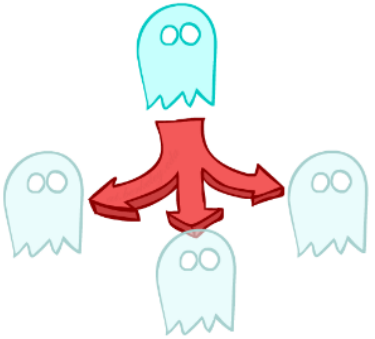


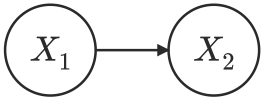

- $\mathbf{P}(\text{Umbrella}_t \mid \text{Rain}_t)$?
- $\mathbf{P}(\text{Rain}_t \mid \text{Umbrella}_{0:t-1})$?
- $\mathbf{P}(\text{Rain}_{t+2} \mid \text{Rain}_t)$?

# Inference tasks

- **Prediction**: $\mathbf{P}(\mathbf{X}_{t+k} \mid \mathbf{e}_{1:t})$ for $k > 0$
  - Computing the posterior distribution over future states.
  - Used for evaluation of possible action sequences.
- **Filtering**: $\mathbf{P}(\mathbf{X}_t \mid \mathbf{e}_{1:t})$
  - Filtering is what a rational agent does to keep track of the current hidden state $\mathbf{X}_t$, its **belief state**, so that rational decisions can be made.
- **Smoothing**: $\mathbf{P}(\mathbf{X}_k \mid \mathbf{e}_{1:t})$ for $0 \leq k < t$
  - Computing the posterior distribution over past states.
  - Used for building better estimates, since it incorporates more evidence.
  - Essential for learning.
- **Most likely explanation**: $\arg\max_{\mathbf{x}_{1:t}} P(\mathbf{x}_{1:t} \mid \mathbf{e}_{1:t})$
  - Decoding with a noisy channel, speech recognition, etc.

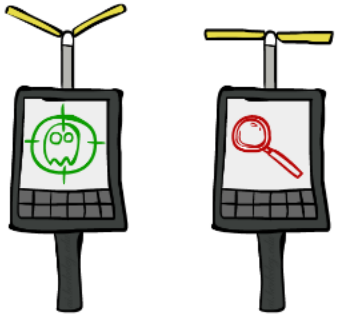
# Base cases

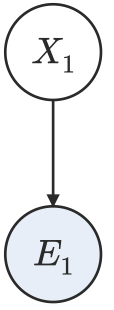





$$\mathbf{P}(\mathbf{X}_2) = \sum_{\mathbf{x}_1} \mathbf{P}(\mathbf{X}_2, \mathbf{x}_1)$$
$$= \sum_{\mathbf{x}_1} P(\mathbf{x}_1)\mathbf{P}(\mathbf{X}_2 \mid \mathbf{x}_1)$$

(Predict) Push $\mathbf{P}(\mathbf{X}_1)$ forward through the transition model.

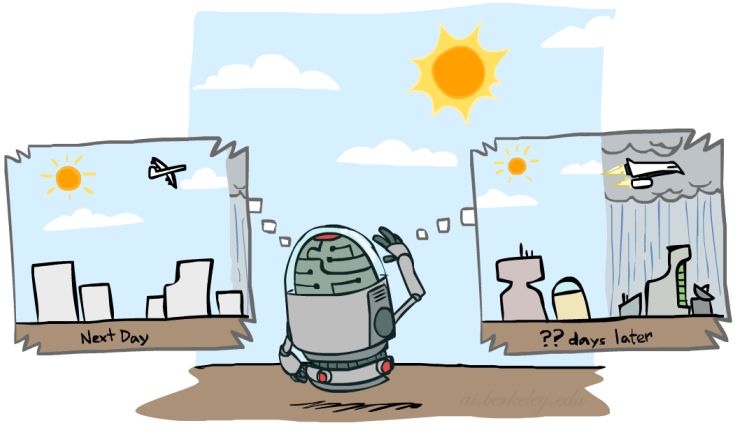





$$\mathbf{P}(\mathbf{X}_1 \mid \mathbf{e}_1) = \frac{\mathbf{P}(\mathbf{e}_1 \mid \mathbf{X}_1)\mathbf{P}(\mathbf{X}_1)}{P(\mathbf{e}_1)}$$
$$\propto \mathbf{P}(\mathbf{e}_1 \mid \mathbf{X}_1)\mathbf{P}(\mathbf{X}_1)$$

(Update) Update $\mathbf{P}(\mathbf{X}_1)$ with the evidence $\mathbf{e}_1$, given the sensor model.

# Prediction




To predict the future $\mathbf{P}(\mathbf{X}_{t+k} \mid \mathbf{e}_{1:t})$:

- **Push** the prior belief state $\mathbf{P}(\mathbf{X}_t \mid \mathbf{e}_{1:t})$ through the transition model:

$$\mathbf{P}(\mathbf{X}_{t+1} \mid \mathbf{e}_{1:t}) = \sum_{\mathbf{x}_t} \mathbf{P}(\mathbf{X}_{t+1} \mid \mathbf{x}_t) P(\mathbf{x}_t \mid \mathbf{e}_{1:t})$$

- Repeat up to $t + k$, using $\mathbf{P}(\mathbf{X}_{t+k-1} \mid \mathbf{e}_{1:t})$ to compute $\mathbf{P}(\mathbf{X}_{t+k} \mid \mathbf{e}_{1:t})$.



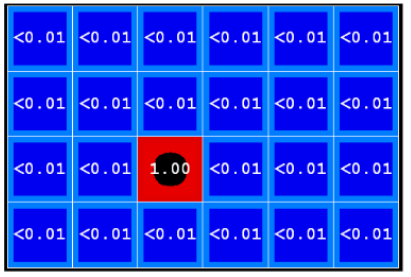

Random dynamics



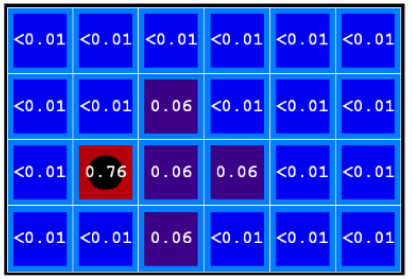

Circular dynamics




Whirlpool dynamics

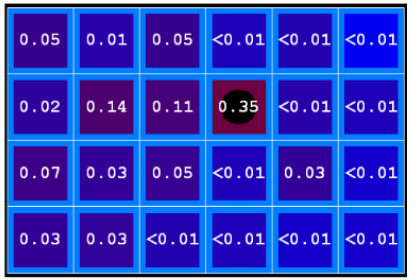



T = 1

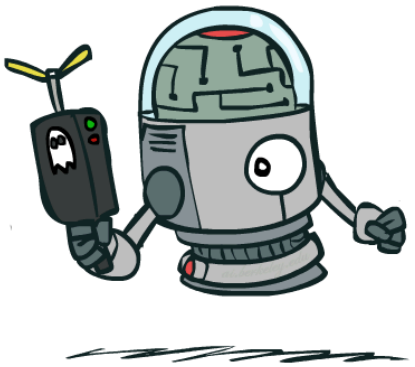


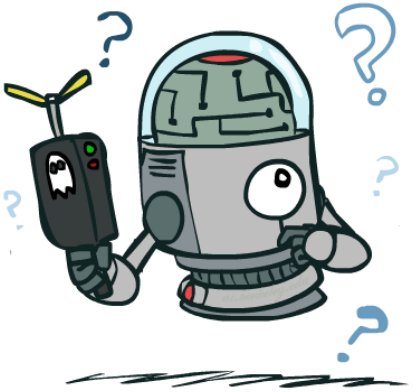

T = 2

...




T = 5







As time passes, uncertainty (usually) increases in the absence of new evidence.

# Filtering

We want to compute the belief state $\mathbf{P}(\mathbf{X}_t \mid \mathbf{e}_{1:t})$ and maintain it as new evidence $\mathbf{e}_{t+1}$ arrives.

The **Bayes filter** alternates two steps:

- (Predict step): Project the belief state from $t$ to $t+1$ through the transition model.
- (Update step): Update it with the evidence $\mathbf{e}_{t+1}$.

The update step is the sequential Bayes update of Lecture 4. What is new is the predict step, since the ghost moves between two readings.



Formally, the Bayes filter is defined as

$$\begin{aligned}
\mathbf{P}(\mathbf{X}_{t+1} \mid \mathbf{e}_{1:t+1}) &= \mathbf{P}(\mathbf{X}_{t+1} \mid \mathbf{e}_{1:t}, \mathbf{e}_{t+1}) \\
&\propto \mathbf{P}(\mathbf{e}_{t+1} \mid \mathbf{X}_{t+1}, \mathbf{e}_{1:t}) \mathbf{P}(\mathbf{X}_{t+1} \mid \mathbf{e}_{1:t}) \\
&\propto \mathbf{P}(\mathbf{e}_{t+1} \mid \mathbf{X}_{t+1}) \mathbf{P}(\mathbf{X}_{t+1} \mid \mathbf{e}_{1:t}) \\
&\propto \mathbf{P}(\mathbf{e}_{t+1} \mid \mathbf{X}_{t+1}) \sum_{\mathbf{x}_t} \mathbf{P}(\mathbf{X}_{t+1} \mid \mathbf{x}_t, \mathbf{e}_{1:t}) P(\mathbf{x}_t \mid \mathbf{e}_{1:t}) \\
&\propto \mathbf{P}(\mathbf{e}_{t+1} \mid \mathbf{X}_{t+1}) \sum_{\mathbf{x}_t} \mathbf{P}(\mathbf{X}_{t+1} \mid \mathbf{x}_t) P(\mathbf{x}_t \mid \mathbf{e}_{1:t})
\end{aligned}$$

where

- the normalization constant

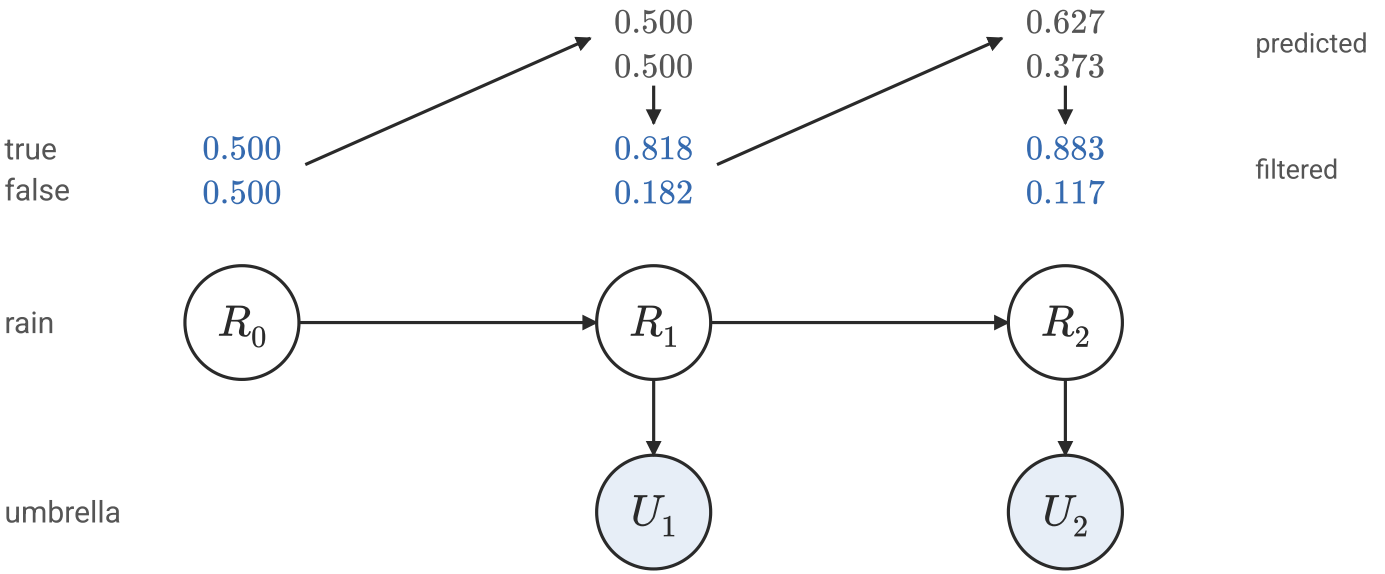
  $$Z = P(\mathbf{e}_{t+1} \mid \mathbf{e}_{1:t}) = \sum_{\mathbf{x}_{t+1}} P(\mathbf{e}_{t+1} \mid \mathbf{x}_{t+1}) P(\mathbf{x}_{t+1} \mid \mathbf{e}_{1:t})$$

  makes the probabilities sum to 1 (it is also written $\alpha = 1/Z$);
- in the last expression, the first and second terms are given by the model while the third is obtained recursively.

We can think of $\mathbf{P}(\mathbf{X}_t \mid \mathbf{e}_{1:t})$ as a **message** $\mathbf{f}_{1:t}$ that is propagated **forward** along the sequence, modified by each transition and updated by each new observation.

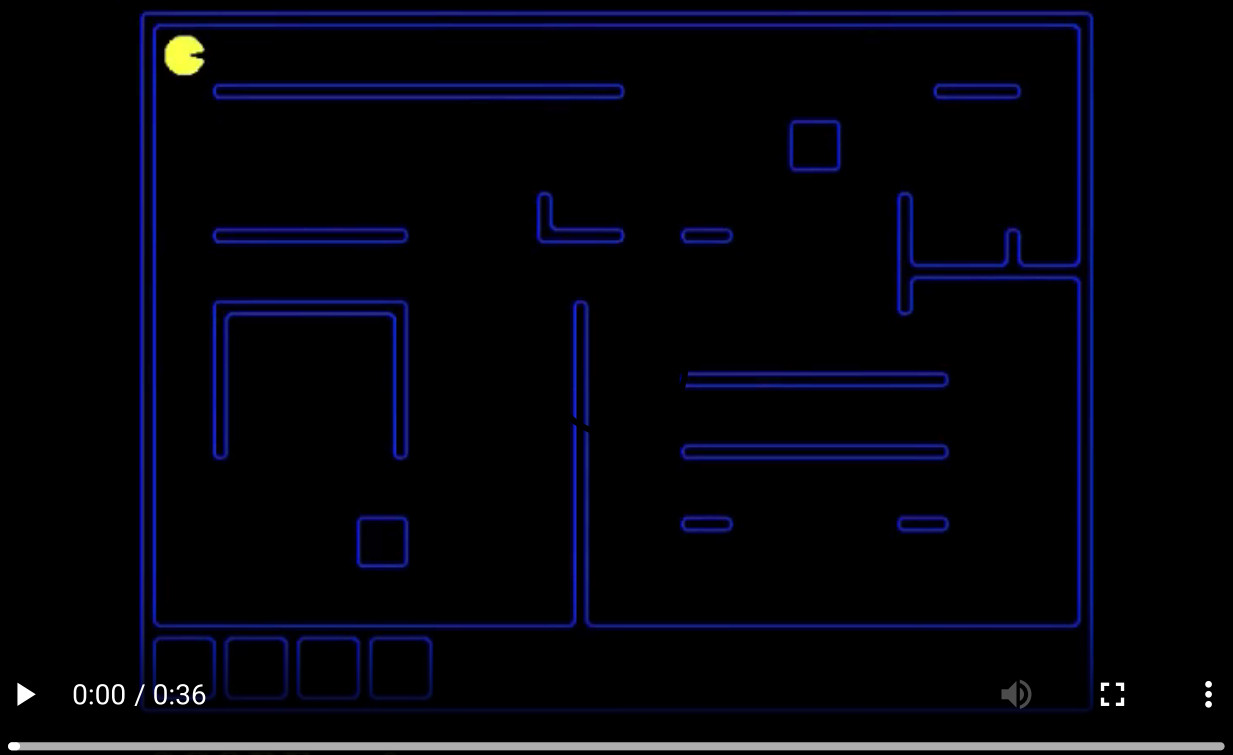
Thus, the process can be implemented as $\mathbf{f}_{1:t+1} \propto \text{forward}(\mathbf{f}_{1:t}, \mathbf{e}_{t+1})$. Its complexity is constant (in time and space) with $t$.

# Example




| $R_{t-1}$ | $P(R_t)$ |
|---|---|
| true | 0.7 |
| false | 0.3 |

| $R_t$ | $P(U_t)$ |
|---|---|
| true | 0.9 |
| false | 0.2 |




Ghostbusters with a Bayes filter

# Smoothing

We want to compute $\mathbf{P}(\mathbf{X}_k \mid \mathbf{e}_{1:t})$ for $0 \leq k < t$.

Dividing the evidence $\mathbf{e}_{1:t}$ into $\mathbf{e}_{1:k}$ and $\mathbf{e}_{k+1:t}$, we have

$$\begin{aligned}\mathbf{P}(\mathbf{X}_k \mid \mathbf{e}_{1:t}) &= \mathbf{P}(\mathbf{X}_k \mid \mathbf{e}_{1:k}, \mathbf{e}_{k+1:t}) \\ &\propto \mathbf{P}(\mathbf{X}_k \mid \mathbf{e}_{1:k})\mathbf{P}(\mathbf{e}_{k+1:t} \mid \mathbf{X}_k, \mathbf{e}_{1:k}) \\ &\propto \mathbf{P}(\mathbf{X}_k \mid \mathbf{e}_{1:k})\mathbf{P}(\mathbf{e}_{k+1:t} \mid \mathbf{X}_k).\end{aligned}$$

Let the **backward** message $\mathbf{b}_{k+1:t}$ correspond to $\mathbf{P}(\mathbf{e}_{k+1:t} \mid \mathbf{X}_k)$. Then,

$$\mathbf{P}(\mathbf{X}_k \mid \mathbf{e}_{1:t}) = \frac{1}{Z}\mathbf{f}_{1:k} \times \mathbf{b}_{k+1:t},$$

where $\times$ is a pointwise multiplication of vectors.

This backward message can be computed using backwards recursion:

$$\begin{aligned}
\mathbf{P}(\mathbf{e}_{k+1:t} \mid \mathbf{X}_k) &= \sum_{\mathbf{x}_{k+1}} \mathbf{P}(\mathbf{e}_{k+1:t} \mid \mathbf{X}_k, \mathbf{x}_{k+1}) \mathbf{P}(\mathbf{x}_{k+1} \mid \mathbf{X}_k) \\
&= \sum_{\mathbf{x}_{k+1}} P(\mathbf{e}_{k+1:t} \mid \mathbf{x}_{k+1}) \mathbf{P}(\mathbf{x}_{k+1} \mid \mathbf{X}_k) \\
&= \sum_{\mathbf{x}_{k+1}} P(\mathbf{e}_{k+1} \mid \mathbf{x}_{k+1}) P(\mathbf{e}_{k+2:t} \mid \mathbf{x}_{k+1}) \mathbf{P}(\mathbf{x}_{k+1} \mid \mathbf{X}_k).
\end{aligned}$$

The first and last factors are given by the model. The second factor is obtained recursively. Therefore,

$$\mathbf{b}_{k+1:t} = \mathbf{backward}(\mathbf{b}_{k+2:t}, \mathbf{e}_{k+1}).$$

# Forward-backward algorithm

**function** FORWARD-BACKWARD($e$, $prior$) **returns** the smoothed distributions

$f[0] \leftarrow prior$, $b \leftarrow$ an all-one vector

**for** $i$ = 1 **to** $t$ **do**

$f[i] \leftarrow \text{FORWARD}(f[i - 1], e[i])$

**for** $i$ = $t$ **downto** 1 **do**

$s[i] \leftarrow \text{NORMALIZE}(f[i] \times b)$

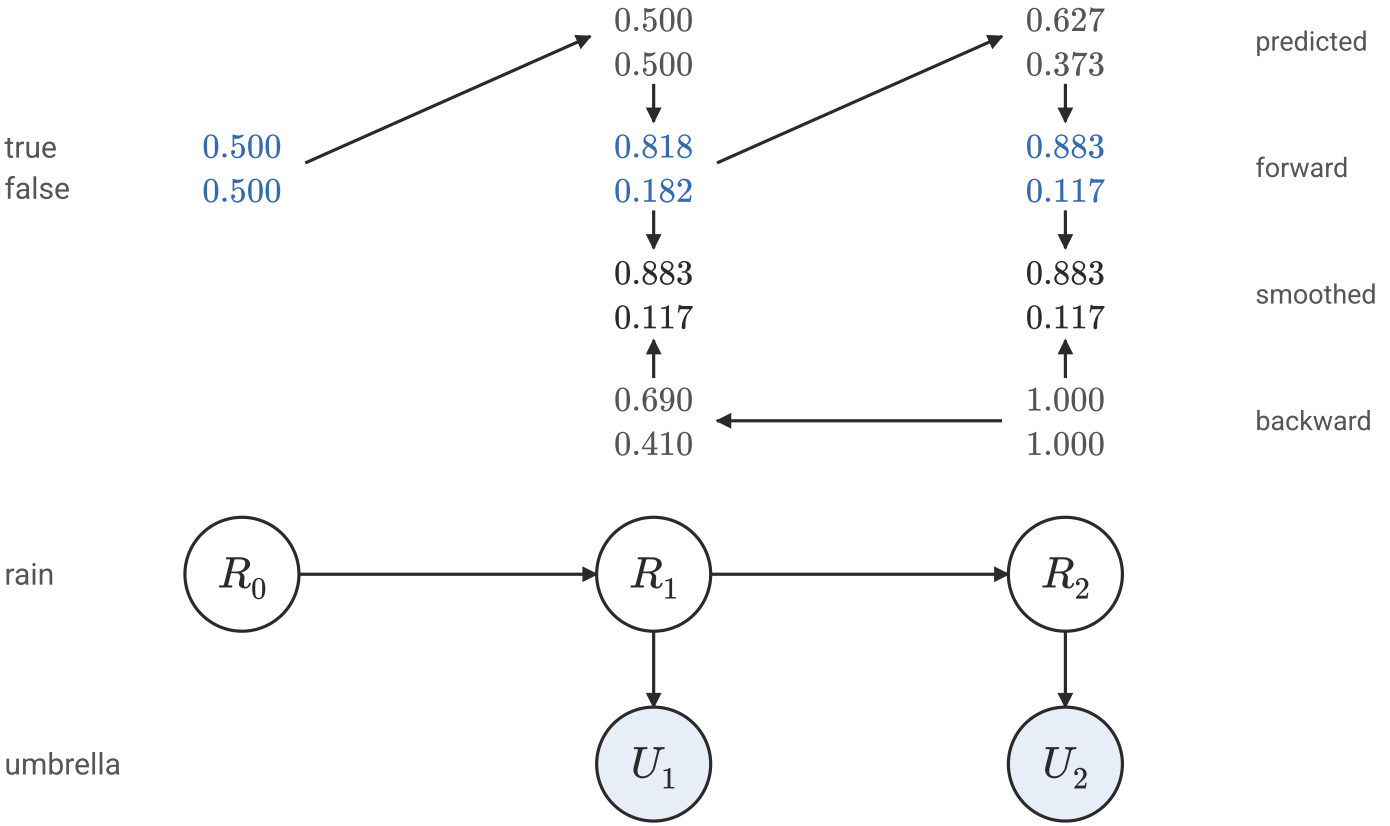
$b \leftarrow \text{BACKWARD}(b, e[i])$

**return** $s$

Complexity:

- Smoothing for a particular time step $k$ takes: $O(t)$
- Smoothing a whole sequence (because of caching): $O(t)$

# Example


predicted
forward
smoothed
backward
true
false
0.500
0.500
0.500
0.500
0.818
0.182
0.883
0.117
0.883
0.117
0.690
0.410
1.000
1.000
0.627
0.373
0.883
0.117
rain
umbrella
$R_0$
$R_1$
$R_2$
$U_1$
$U_2$

# Most likely explanation







Suppose that $[\mathbf{true}, \mathbf{true}, \mathbf{false}, \mathbf{true}, \mathbf{true}]$ is the umbrella sequence.

- What is the weather sequence that is the most likely to explain this?
- Among all $2^5$ sequences, is there an (efficient) way to find the most likely one?

The most likely sequence **is not** the sequence of the most likely states!

The most likely path to each $\mathbf{x}_{t+1}$, is the most likely path to **some** $\mathbf{x}_t$ plus one more step. Therefore,

$$\begin{aligned}&\max_{\mathbf{x}_{1:t}} \mathbf{P}(\mathbf{x}_{1:t}, \mathbf{X}_{t+1} \mid \mathbf{e}_{1:t+1}) \\ &\propto \mathbf{P}(\mathbf{e}_{t+1} \mid \mathbf{X}_{t+1}) \max_{\mathbf{x}_t}(\mathbf{P}(\mathbf{X}_{t+1} \mid \mathbf{x}_t) \max_{\mathbf{x}_{1:t-1}} \mathbf{P}(\mathbf{x}_{1:t-1}, \mathbf{x}_t \mid \mathbf{e}_{1:t})).\end{aligned}$$

This is identical to filtering, except that

- the forward message $\mathbf{f}_{1:t} = \mathbf{P}(\mathbf{X}_t \mid \mathbf{e}_{1:t})$ is replaced with

$$\mathbf{m}_{1:t} = \max_{\mathbf{x}_{1:t-1}} \mathbf{P}(\mathbf{x}_{1:t-1}, \mathbf{X}_t \mid \mathbf{e}_{1:t}),$$

  where $\mathbf{m}_{1:t}(i)$ gives the probability of the most likely path to state $i$.

- The update has its sum replaced by max.

The resulting algorithm is called the **Viterbi algorithm**, which computes the most likely explanation as

$$\mathbf{m}_{1:t+1} \propto \mathbf{P}(\mathbf{e}_{t+1} \mid \mathbf{X}_{t+1}) \max_{\mathbf{x}_t} \mathbf{P}(\mathbf{X}_{t+1} \mid \mathbf{x}_t) \mathbf{m}_{1:t}.$$

Its complexity is linear in $t$, the length of the sequence.

# Example

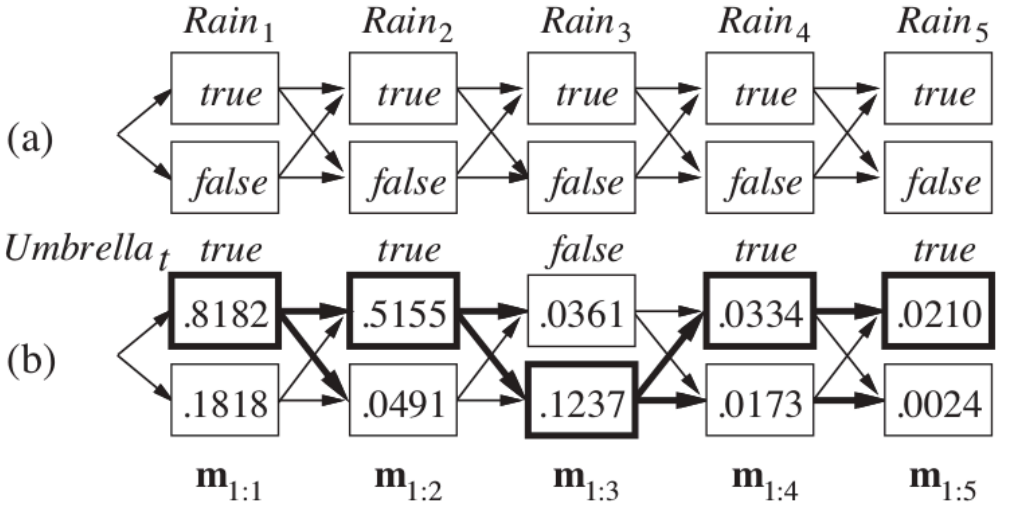



**Figure 15.5** (a) Possible state sequences for $Rain_t$ can be viewed as paths through a graph of the possible states at each time step. (States are shown as rectangles to avoid confusion with nodes in a Bayes net.) (b) Operation of the Viterbi algorithm for the umbrella observation sequence $[true, true, false, true, true]$. For each $t$, we have shown the values of the message $\mathbf{m}_{1:t}$, which gives the probability of the best sequence reaching each state at time $t$. Also, for each state, the bold arrow leading into it indicates its best predecessor as measured by the product of the preceding sequence probability and the transition probability. Following the bold arrows back from the most likely state in $\mathbf{m}_{1:5}$ gives the most likely sequence.

# Hidden Markov models

So far, we described Markov processes over arbitrary sets of state variables $\mathbf{X}_t$ and evidence variables $\mathbf{E}_t$.

- A **hidden Markov model** (HMM) is a Markov process in which the state $\mathbf{X}_t$ and the evidence $\mathbf{E}_t$ are both **single discrete** random variables.
  - $\mathbf{X}_t = X_t$, with domain $D_{X_t} = \{1, ..., S\}$
  - $\mathbf{E}_t = E_t$, with domain $D_{E_t} = \{1, ..., R\}$
- This restricted structure allows for a reformulation of the forward-backward algorithm in terms of matrix-vector operations.

## Note on terminology

Some authors instead divide Markov models into two classes, depending on the observability of the system state:

- Observable system state: Markov chains
- Partially-observable system state: Hidden Markov models.

We follow here instead the terminology of the textbook, as defined in the previous slide.

## Simplified matrix algorithms

- The prior $\mathbf{P}(X_0)$ becomes a (normalized) column vector $\mathbf{f}_0 \in \mathbb{R}_+^S$.
- The transition model $\mathbf{P}(X_t \mid X_{t-1})$ becomes an $S \times S$ **transition matrix** $\mathbf{T}$, such that

$$\mathbf{T}_{ij} = P(X_t = j \mid X_{t-1} = i).$$

- The sensor model $\mathbf{P}(E_t \mid X_t)$ is defined as an $S \times R$ **sensor matrix** $\mathbf{B}$, such that

$$\mathbf{B}_{ij} = P(E_t = j \mid X_t = i).$$

- Let the observation matrix $\mathbf{O}_t$ be a diagonal matrix whose elements corresponds to the column $e_t$ of the sensor matrix $\mathbf{B}$.
- If we use column vectors to represent forward and backward messages, then we have

$$\mathbf{f}_{1:t+1} = \frac{1}{Z}\mathbf{O}_{t+1}\mathbf{T}^T\mathbf{f}_{1:t}$$

$$\mathbf{b}_{k+1:t} = \mathbf{T}\mathbf{O}_{k+1}\mathbf{b}_{k+2:t},$$

  where $\mathbf{b}_{t+1:t}$ is an all-one vector of size $S$.
- Therefore the forward-backward algorithm needs time $O(S^2t)$ and space $O(St)$.

## Example

Suppose that $[\mathbf{true}, \mathbf{true}, \mathbf{false}, \mathbf{true}, \mathbf{true}]$ is the umbrella sequence.

$$\mathbf{f}_0 = \begin{pmatrix} 0.5 \\ 0.5 \end{pmatrix}$$

$$\mathbf{T} = \begin{pmatrix} 0.7 & 0.3 \\ 0.3 & 0.7 \end{pmatrix}$$

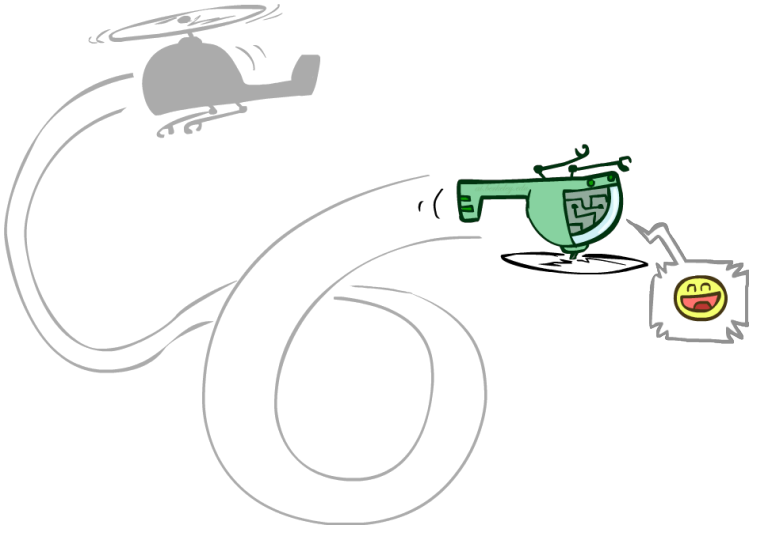
$$\mathbf{B} = \begin{pmatrix} 0.9 & 0.1 \\ 0.2 & 0.8 \end{pmatrix}$$

$$\mathbf{O}_1 = \mathbf{O}_2 = \mathbf{O}_4 = \mathbf{O}_5 = \begin{pmatrix} 0.9 & 0.0 \\ 0.0 & 0.2 \end{pmatrix}$$

$$\mathbf{O}_3 = \begin{pmatrix} 0.1 & 0.0 \\ 0.0 & 0.8 \end{pmatrix}$$

See `demo/lec6/forward-backward.ipynb` for the execution.

# Filters



Suppose we want to track the position and velocity of a robot from noisy observations collected over time.

Formally, we want to estimate **continuous** state variables such as

- the position $\mathbf{X}_t$ of the robot at time $t$,
- the velocity $\dot{\mathbf{X}}_t$ of the robot at time $t$.

We assume **discrete** time steps.

# Continuous variables

Let $X : \Omega \to D_X$ be a random variable.

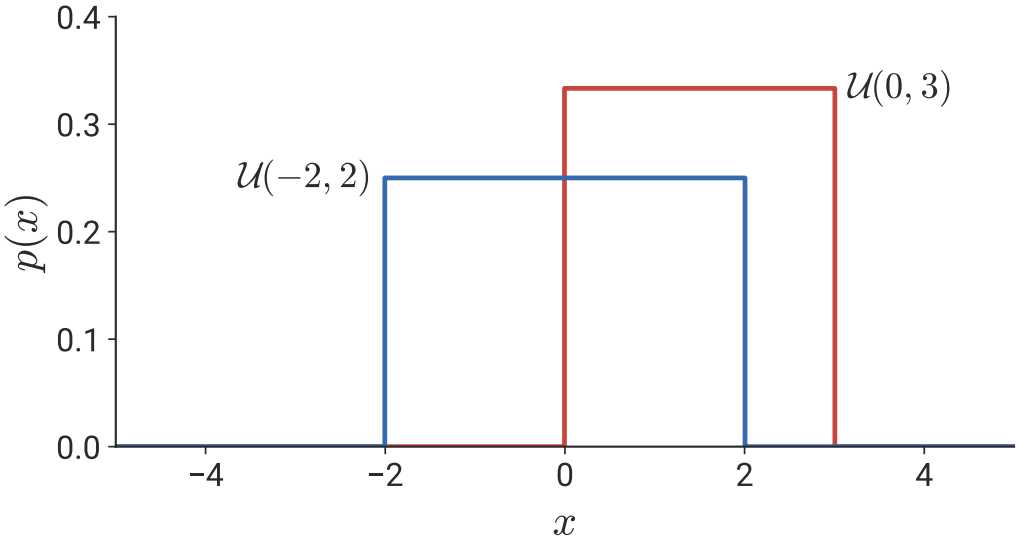
- When $D_X$ is uncountably infinite (e.g., $D_X = \mathbb{R}$), $X$ is called a **continuous random variable**.
- If $X$ is absolutely continuous, its probability distribution is described by a **density function** $p$ that assigns a probability to any interval $[a, b] \subseteq D_X$ such that

$$P(a < X \leq b) = \int_a^b p(x)dx,$$

where $p$ is non-negative piecewise continuous and such that

$$\int_{D_X} p(x)dx = 1.$$

## Uniform

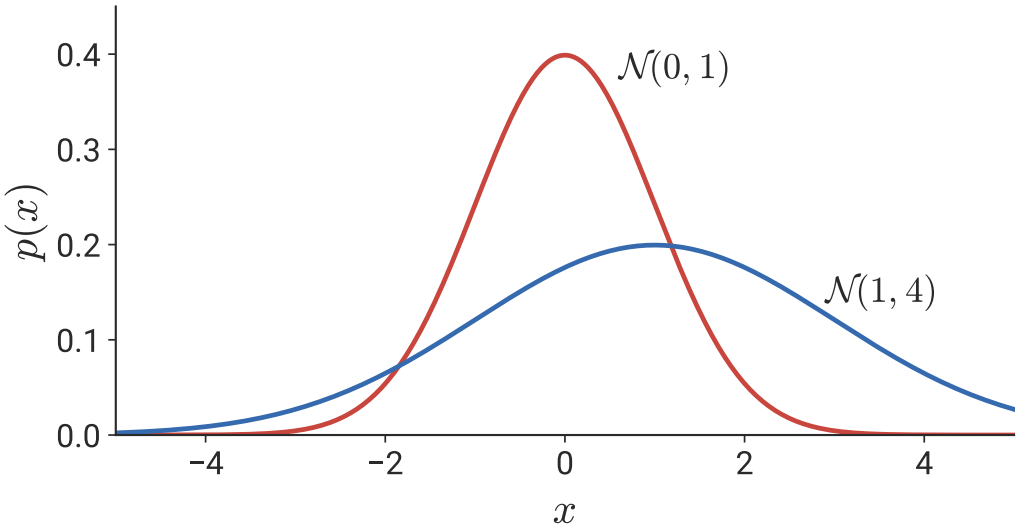



The uniform distribution $\mathcal{U}(a, b)$ is described by the density function

$$p(x) = \begin{cases} \frac{1}{b-a} & \text{if } x \in [a, b] \\ 0 & \text{otherwise} \end{cases}$$

where $a \in \mathbb{R}$ and $b \in \mathbb{R}$ are the bounds of its support.

## Gaussian

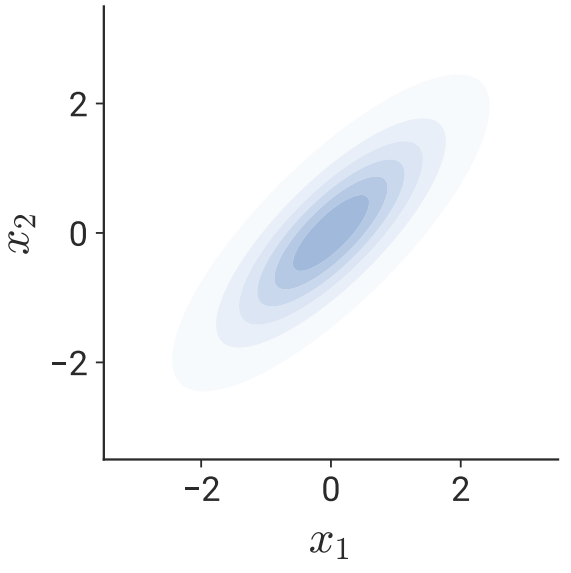



The Gaussian (or Normal) distribution $\mathcal{N}(\mu, \sigma^2)$ is described by the density function

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

where $\mu \in \mathbb{R}$ is its mean and $\sigma^2 \in \mathbb{R}^+$ its variance.

# Multivariate Gaussian




The multivariate Gaussian distribution generalizes to $n$ random variables. Its (joint) density function is defined as

$$p(\mathbf{x} = x_1, ..., x_n) = \frac{1}{\sqrt{(2\pi)^n |\mathbf{\Sigma}|}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^T \mathbf{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu})\right)$$

where $\boldsymbol{\mu} \in \mathbb{R}^n$ is the mean vector and $\mathbf{\Sigma} \in \mathbb{R}^{n \times n}$ is the covariance matrix.

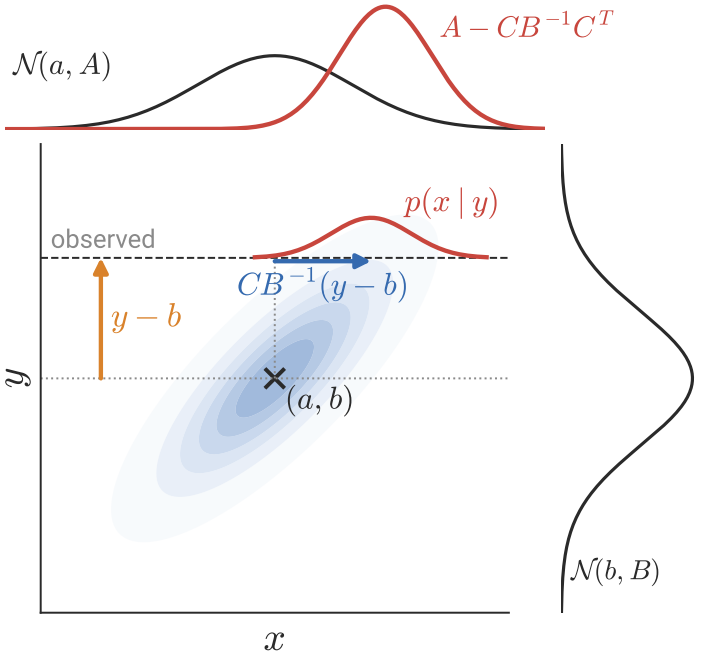
## Cheat sheet for Gaussian distributions (Särkkä, 2013)

If $\mathbf{x}$ and $\mathbf{y}$ have the joint Gaussian distribution

$$p\begin{pmatrix}\mathbf{x}\\ \mathbf{y}\end{pmatrix} = \mathcal{N}\left(\begin{pmatrix}\mathbf{x}\\ \mathbf{y}\end{pmatrix} \middle| \begin{pmatrix}\mathbf{a}\\ \mathbf{b}\end{pmatrix}, \begin{pmatrix}\mathbf{A} & \mathbf{C}\\ \mathbf{C}^T & \mathbf{B}\end{pmatrix}\right),$$

then the marginal and conditional distributions of $\mathbf{x}$ and $\mathbf{y}$ are given by

$$\begin{aligned}
p(\mathbf{x}) &= \mathcal{N}(\mathbf{x} \mid \mathbf{a}, \mathbf{A})\\
p(\mathbf{y}) &= \mathcal{N}(\mathbf{y} \mid \mathbf{b}, \mathbf{B})\\
p(\mathbf{x} \mid \mathbf{y}) &= \mathcal{N}(\mathbf{x} \mid \mathbf{a} + \mathbf{C}\mathbf{B}^{-1}(\mathbf{y} - \mathbf{b}), \mathbf{A} - \mathbf{C}\mathbf{B}^{-1}\mathbf{C}^T)\\
p(\mathbf{y} \mid \mathbf{x}) &= \mathcal{N}(\mathbf{y} \mid \mathbf{b} + \mathbf{C}^T\mathbf{A}^{-1}(\mathbf{x} - \mathbf{a}), \mathbf{B} - \mathbf{C}^T\mathbf{A}^{-1}\mathbf{C}).
\end{aligned}$$




Observing $\mathbf{y}$ moves the guess $\mathbf{a}$ in proportion to the surprise $\mathbf{y} - \mathbf{b}$, and shrinks the covariance $\mathbf{A}$:

$$\mathbf{a} + \mathbf{C}\mathbf{B}^{-1}(\mathbf{y} - \mathbf{b}), \qquad \mathbf{A} - \mathbf{C}\mathbf{B}^{-1}\mathbf{C}^T.$$

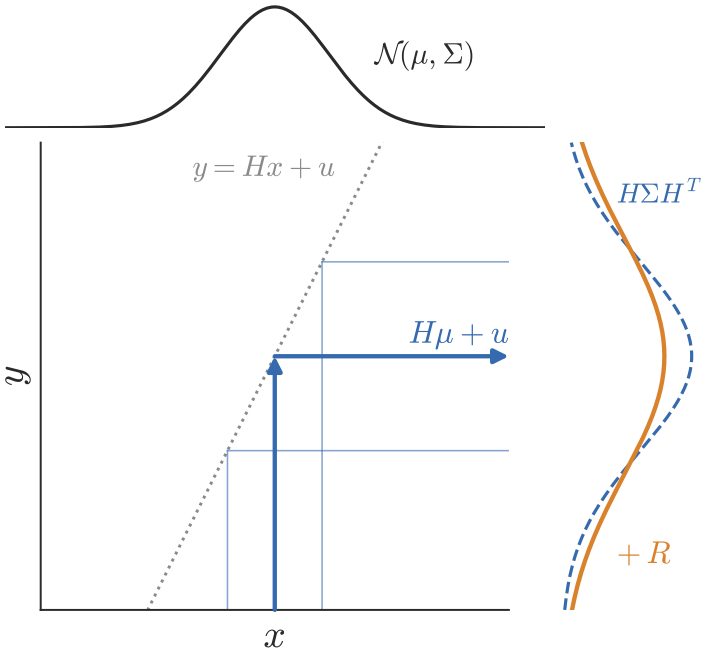
The larger the covariance $\mathbf{C}$ between $\mathbf{x}$ and $\mathbf{y}$, the larger the move and the shrinkage.

If the random variables $\mathbf{x}$ and $\mathbf{y}$ have Gaussian probability distributions

$$p(\mathbf{x}) = \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma})$$
$$p(\mathbf{y} \mid \mathbf{x}) = \mathcal{N}(\mathbf{y} \mid \mathbf{H}\mathbf{x} + \mathbf{u}, \mathbf{R}),$$

then the joint distribution of $\mathbf{x}$ and $\mathbf{y}$ is Gaussian with

$$p\begin{pmatrix}\mathbf{x}\\ \mathbf{y}\end{pmatrix} = \mathcal{N}\left(\begin{pmatrix}\mathbf{x}\\ \mathbf{y}\end{pmatrix} \middle| \begin{pmatrix}\boldsymbol{\mu}\\ \mathbf{H}\boldsymbol{\mu} + \mathbf{u}\end{pmatrix}, \begin{pmatrix}\boldsymbol{\Sigma} & \boldsymbol{\Sigma}\mathbf{H}^T\\ \mathbf{H}\boldsymbol{\Sigma} & \mathbf{H}\boldsymbol{\Sigma}\mathbf{H}^T + \mathbf{R}\end{pmatrix}\right).$$




Equivalently, $\mathbf{y = Hx + u + v}$, with a noise $\mathbf{v \sim \mathcal{N}(0, R)}$ independent of $\mathbf{x}$.
The linear map sends the mean $\boldsymbol{\mu}$ to $\mathbf{H\mu + u}$ and the covariance $\mathbf{\Sigma}$ to $\mathbf{H\Sigma H}^T$.
The noise being independent, its covariance $\mathbf{R}$ adds to it.

# Continuous Bayes filter

The Bayes filter extends to **continuous** state and evidence variables $\mathbf{X}_t$ and $\mathbf{E}_t$.

The rules of Lecture 4 are unchanged: only the summations are replaced with integrals and the probability mass functions with densities, which gives the recursive Bayesian relation

$$p(\mathbf{x}_{t+1} \mid \mathbf{e}_{1:t+1}) \propto p(\mathbf{e}_{t+1} \mid \mathbf{x}_{t+1}) \int p(\mathbf{x}_{t+1} \mid \mathbf{x}_t) p(\mathbf{x}_t \mid \mathbf{e}_{1:t}) d\mathbf{x}_t,$$

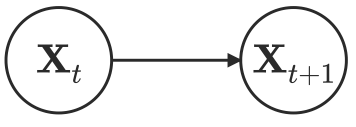
where the normalization constant is

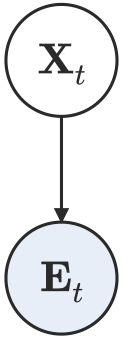
$$Z = \int p(\mathbf{e}_{t+1} \mid \mathbf{x}_{t+1}) p(\mathbf{x}_{t+1} \mid \mathbf{e}_{1:t}) d\mathbf{x}_{t+1}.$$

# Kalman filter

The **Kalman filter** is a special case of the Bayes filter, which assumes:

- Gaussian prior
- Linear Gaussian transition model
- Linear Gaussian sensor model




$$p(\mathbf{x}_{t+1} \mid \mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t+1} \mid \mathbf{A}\mathbf{x}_t + \mathbf{b}, \mathbf{Q})$$

Transition model

$$p(\mathbf{e}_t \mid \mathbf{x}_t) = \mathcal{N}(\mathbf{e}_t \mid \mathbf{H}\mathbf{x}_t + \mathbf{d}, \mathbf{R})$$

Sensor model

## 1d Gaussian random walk

- Gaussian prior:

$$p(x_0) = \mathcal{N}(x_0 \mid \mu_0, \sigma_0^2)$$

- The transition model adds random perturbations of constant variance:

$$p(x_{t+1} \mid x_t) = \mathcal{N}(x_{t+1} \mid x_t, \sigma_x^2)$$

- The sensor model yields measurements with Gaussian noise of constant variance:

$$p(e_t \mid x_t) = \mathcal{N}(e_t \mid x_t, \sigma_e^2)$$

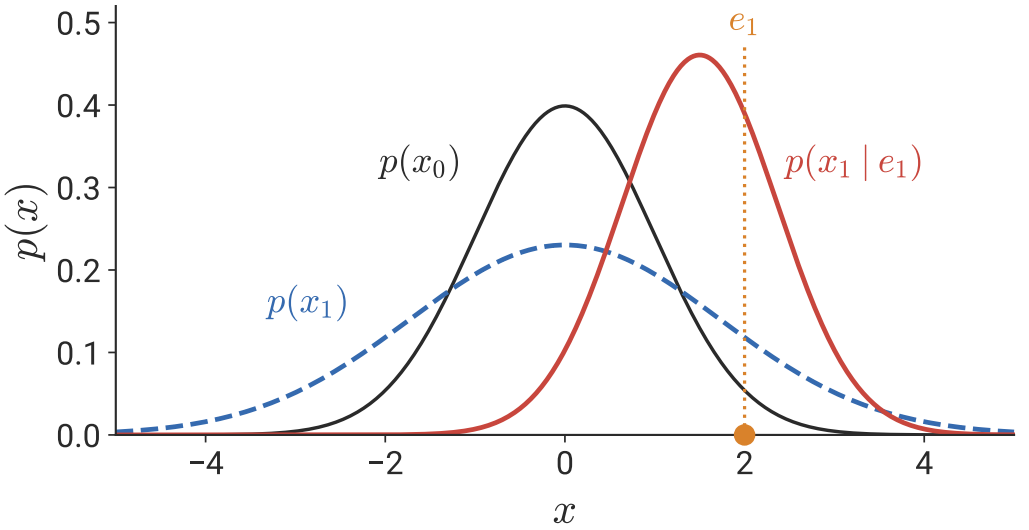
The one-step predicted distribution is given by

$$
\begin{aligned}
p(x_1) &= \int p(x_1 \mid x_0) p(x_0) dx_0 \\
&\propto \int \exp\left(-\frac{1}{2}\frac{(x_1 - x_0)^2}{\sigma_x^2}\right) \exp\left(-\frac{1}{2}\frac{(x_0 - \mu_0)^2}{\sigma_0^2}\right) dx_0 \\
&\propto \int \exp\left(-\frac{1}{2}\frac{\sigma_0^2(x_1 - x_0)^2 + \sigma_x^2(x_0 - \mu_0)^2}{\sigma_0^2 \sigma_x^2}\right) dx_0 \\
&\dots \text{ (simplify by completing the square)} \\
&\propto \exp\left(-\frac{1}{2}\frac{(x_1 - \mu_0)^2}{\sigma_0^2 + \sigma_x^2}\right) \\
&= \mathcal{N}(x_1 \mid \mu_0, \sigma_0^2 + \sigma_x^2)
\end{aligned}
$$

Note that the same result can be obtained by using instead the Gaussian models identities.

For the update step, we need to condition on the observation at the first time step:

$$
\begin{aligned}
p(x_1 \mid e_1) &\propto p(e_1 \mid x_1) p(x_1) \\
&\propto \exp\left(-\frac{1}{2}\frac{(e_1 - x_1)^2}{\sigma_e^2}\right) \exp\left(-\frac{1}{2}\frac{(x_1 - \mu_0)^2}{\sigma_0^2 + \sigma_x^2}\right) \\
&\propto \exp\left(-\frac{1}{2}\frac{\left(x_1 - \frac{(\sigma_0^2+\sigma_x^2)e_1 + \sigma_e^2\mu_0}{\sigma_0^2+\sigma_x^2+\sigma_e^2}\right)^2}{\frac{(\sigma_0^2+\sigma_x^2)\sigma_e^2}{\sigma_0^2+\sigma_x^2+\sigma_e^2}}\right) \\
&= \mathcal{N}\left(x_1 \middle| \frac{(\sigma_0^2 + \sigma_x^2)e_1 + \sigma_e^2\mu_0}{\sigma_0^2 + \sigma_x^2 + \sigma_e^2}, \frac{(\sigma_0^2 + \sigma_x^2)\sigma_e^2}{\sigma_0^2 + \sigma_x^2 + \sigma_e^2}\right)
\end{aligned}
$$




In summary, the update equations given a new evidence $e_{t+1}$ are:

$$\mu_{t+1} = \frac{(\sigma_t^2 + \sigma_x^2)e_{t+1} + \sigma_e^2 \mu_t}{\sigma_t^2 + \sigma_x^2 + \sigma_e^2}$$

$$\sigma_{t+1}^2 = \frac{(\sigma_t^2 + \sigma_x^2)\sigma_e^2}{\sigma_t^2 + \sigma_x^2 + \sigma_e^2}$$

## Kalman update equations

The same derivations generalize to multivariate Gaussian distributions. Assuming the transition and sensor models

$$p(\mathbf{x}_{t+1} \mid \mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t+1} \mid \mathbf{A}\mathbf{x}_t, \mathbf{Q})$$
$$p(\mathbf{e}_t \mid \mathbf{x}_t) = \mathcal{N}(\mathbf{e}_t \mid \mathbf{H}\mathbf{x}_t, \mathbf{R}),$$

the prediction step yields

$$\begin{aligned} p(\mathbf{x}_{t+1} \mid \mathbf{e}_{1:t}) &= \int p(\mathbf{x}_{t+1} \mid \mathbf{x}_t) p(\mathbf{x}_t \mid \mathbf{e}_{1:t}) d\mathbf{x}_t \\ &= \mathcal{N}(\mathbf{x}_{t+1} \mid \boldsymbol{\mu}_{t+1}^-, \boldsymbol{\Sigma}_{t+1}^-) \end{aligned}$$

where

$$\begin{aligned} \boldsymbol{\mu}_{t+1}^- &= \mathbf{A}\boldsymbol{\mu}_t \\ \boldsymbol{\Sigma}_{t+1}^- &= \mathbf{A}\boldsymbol{\Sigma}_t\mathbf{A}^T + \mathbf{Q}. \end{aligned}$$

The update step yields the final Kalman filter equations,

$$p(\mathbf{x}_{t+1} \mid \mathbf{e}_{1:t+1}) = \mathcal{N}(\mathbf{x}_{t+1} \mid \boldsymbol{\mu}_{t+1}, \boldsymbol{\Sigma}_{t+1})$$

where

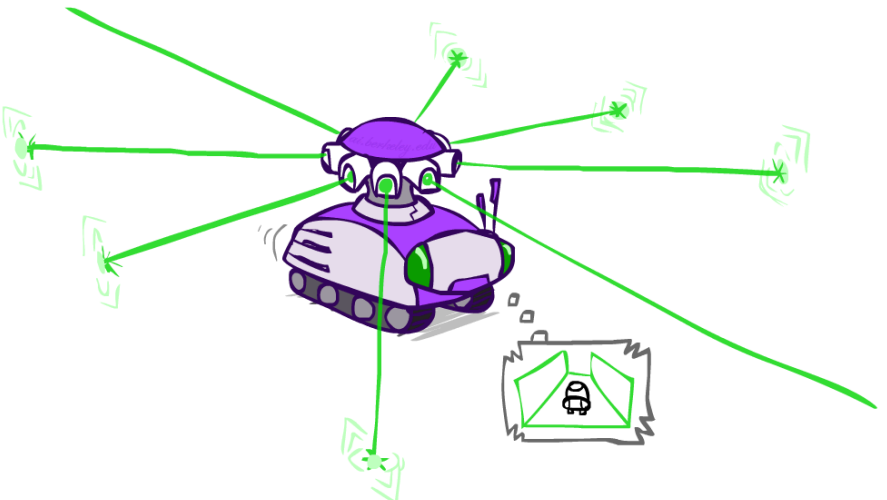
$$\begin{aligned}
\boldsymbol{\mu}_{t+1} &= \boldsymbol{\mu}_{t+1}^{-} + \mathbf{K}_{t+1}(\mathbf{e}_{t+1} - \mathbf{H}\boldsymbol{\mu}_{t+1}^{-}) \\
\boldsymbol{\Sigma}_{t+1} &= (\mathbf{I} - \mathbf{K}_{t+1}\mathbf{H})\boldsymbol{\Sigma}_{t+1}^{-} \\
\mathbf{K}_{t+1} &= \boldsymbol{\Sigma}_{t+1}^{-}\mathbf{H}^{T}(\mathbf{H}\boldsymbol{\Sigma}_{t+1}^{-}\mathbf{H}^{T} + \mathbf{R})^{-1}
\end{aligned}$$

in which $\mathbf{K}_{t+1}$ is called the **Kalman gain** and represents the relative weight given to the new observation versus the prediction.

# Particle filter

When the transition and sensor models are non-linear and/or non-Gaussian, the Kalman filter is not applicable.

The **particle filter** is a sampling-based approximate inference algorithm for general continuous state-space models.

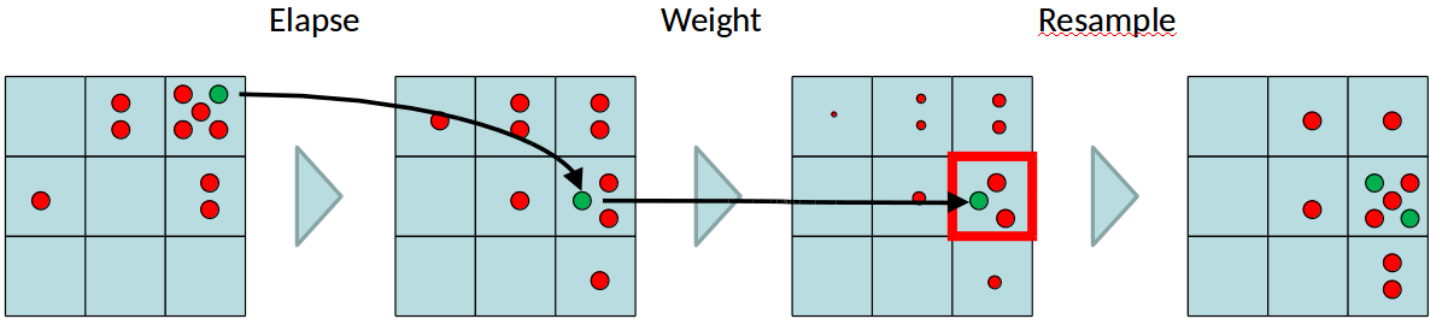


## Core idea

- A particle filter approximates the filtering distribution $p(\mathbf{x}_t \mid \mathbf{e}_{1:t})$ using a set $\{\mathbf{x}_t^i\}$ of samples called **particles**.
- The particles are propagated over time using the transition model.
- The particles are weighted according to the likelihood of the evidence given the particle's state.
- Low-weight particles are discarded, and high-weight particles are duplicated (resampling).

This scales to high dimensions!

# Update cycle



**function** PARTICLE-FILTERING($e$, $N$, $model$) **returns** a set of samples

  **persistent**: $S$, $N$ samples, drawn from the prior

  **for each** $i$ in 1, ..., $N$ **do**

    $S[i] \leftarrow$ a sample from $P(x' \mid x = S[i])$

    $W[i] \leftarrow P(e \mid x = S[i])$

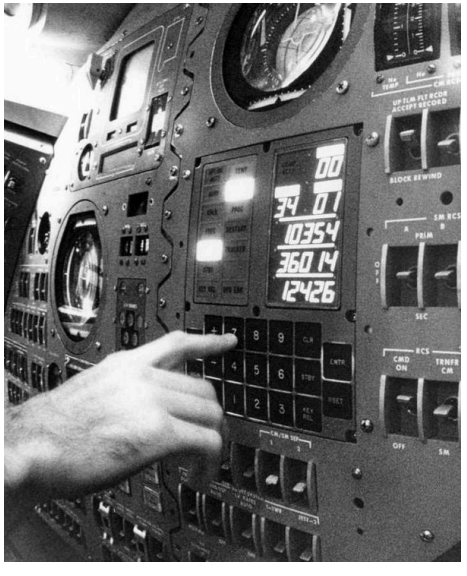
  $S \leftarrow$ WEIGHTED-SAMPLE-WITH-REPLACEMENT($N$, $S$, $W$)

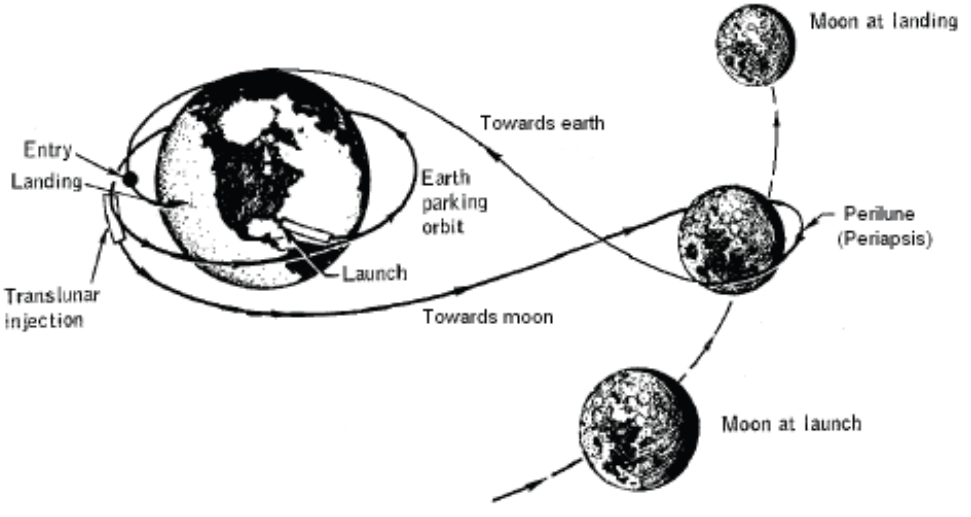
  **return** $S$

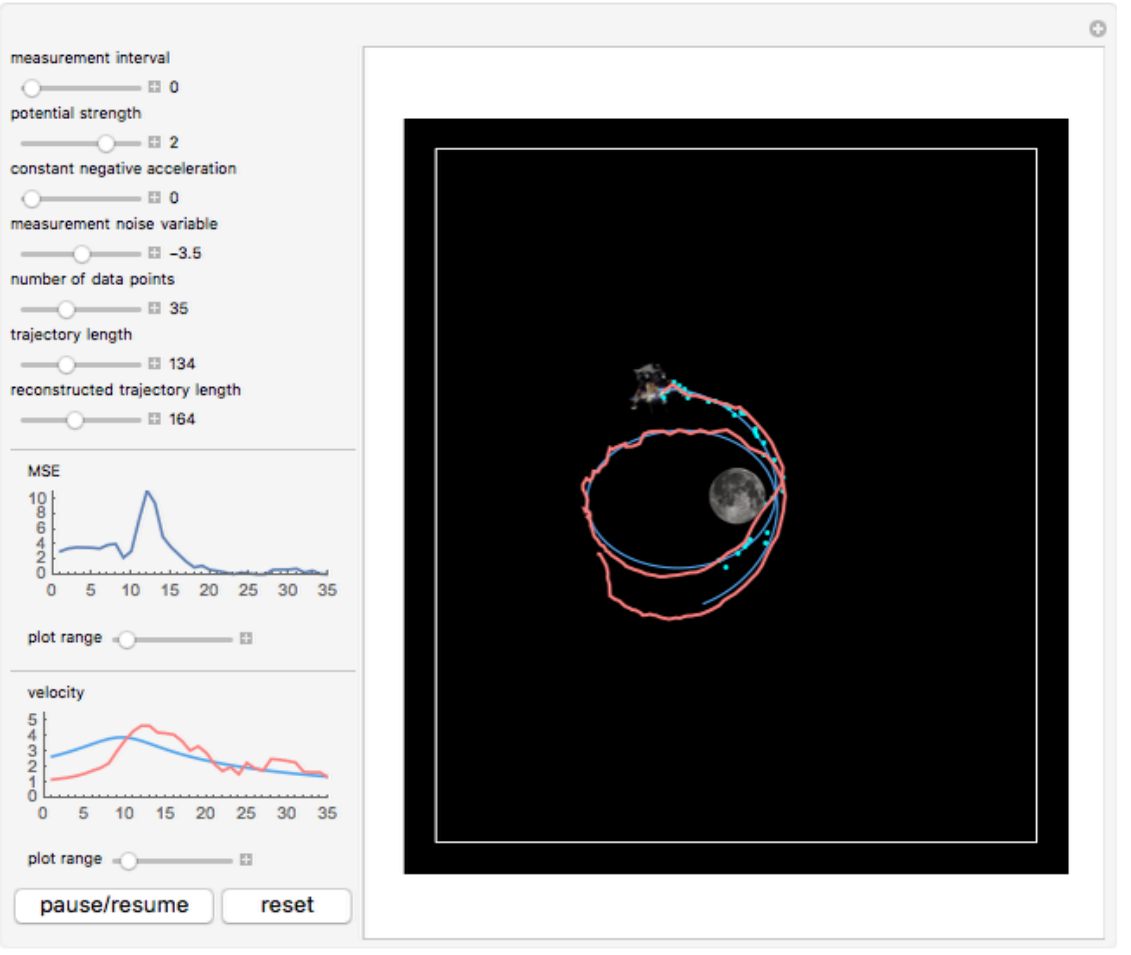
# Applications

# Apollo guidance computer

The Apollo Guidance Computer used a Kalman filter to estimate the position of the spacecraft. The Kalman filter was used to merge new data with past position measurements to produce an optimal position estimate of the spacecraft.



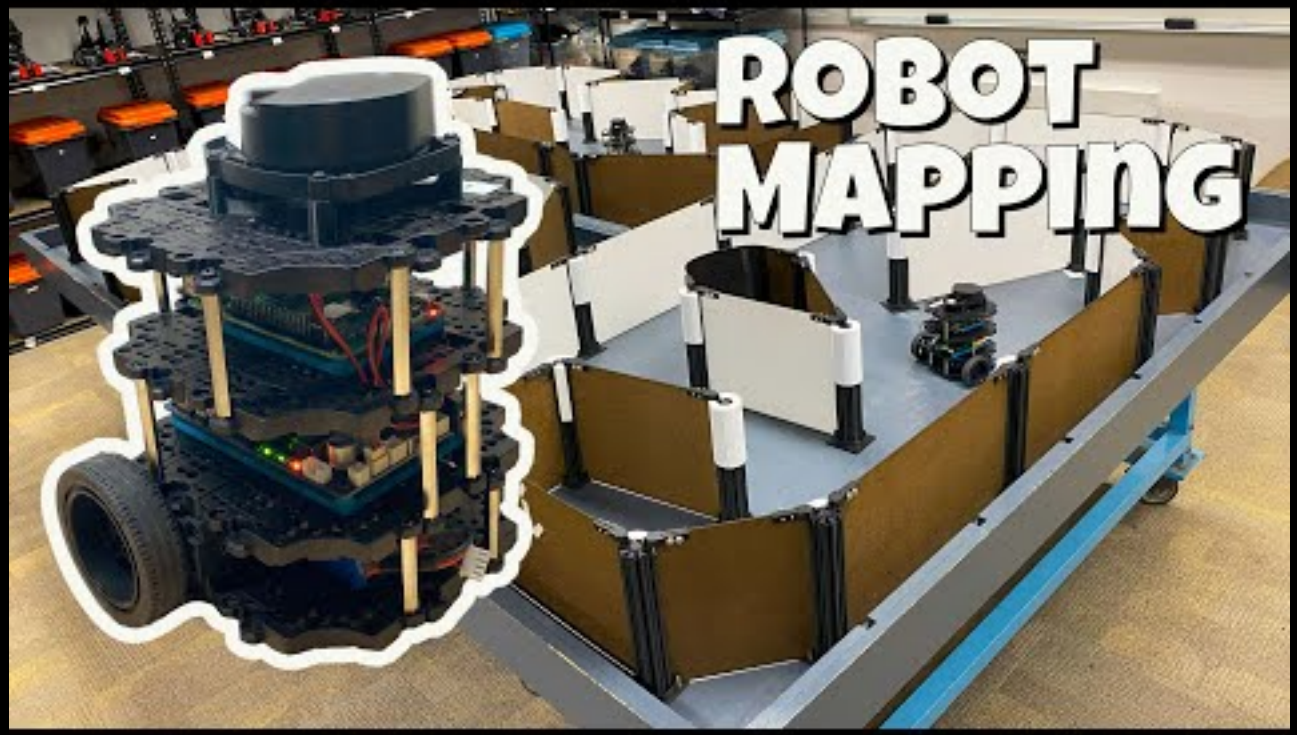


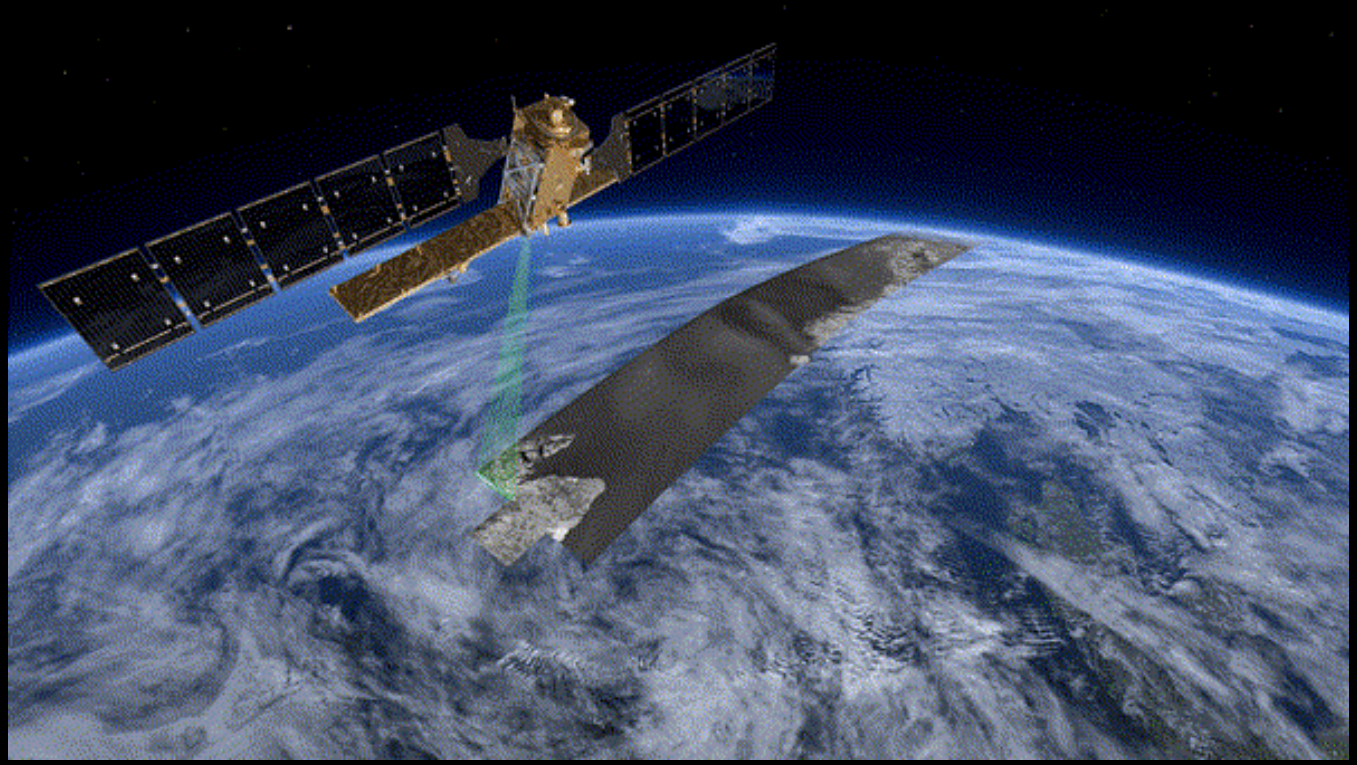



Demo: tracking an object in space using the Kalman Filter.

## Robot localization

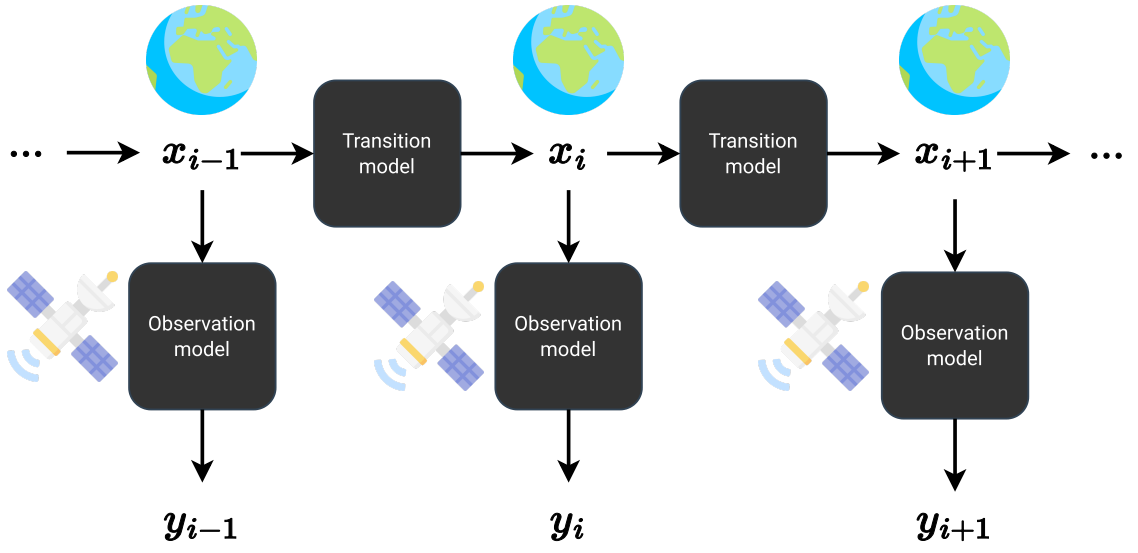
Filtering algorithms such as the Kalman filter and the particle filter are widely used for robot localization, i.e., estimating the position and orientation of a robot based on sensor data and a motion model.





## Data assimilation for weather

Filtering is used to combine observations of the atmosphere with numerical models to estimate its current state and initialize weather forecasts.




Formally, the goal of **data assimilation** is to estimate plausible atmospheric trajectories $x_{1:T}$ given one or more noisy observations $y_{1:T}$ as the posterior

$$p(x_{1:T} \mid y_{1:T}) \propto \prod_{t=1}^{T} p(y_t \mid x_t) p(x_t \mid x_{t-1}),$$

where the transition model $p(x_t \mid x_{t-1})$ is given by a numerical weather prediction model and the sensor model $p(y_t \mid x_t)$ describes the observation process from satellites, radars, weather stations, etc.


© ECMWF

# Summary

- A temporal model replicates the state $\mathbf{X}_t$ and the evidence $\mathbf{E}_t$ over time. Under the Markov and stationarity assumptions, it is specified by a prior $\mathbf{P}(\mathbf{X}_0)$, a transition model $\mathbf{P}(\mathbf{X}_{t+1} \mid \mathbf{X}_t)$ and a sensor model $\mathbf{P}(\mathbf{E}_t \mid \mathbf{X}_t)$.
- **Filtering** maintains the belief state by alternating a predict and an update step,

$$\mathbf{P}(\mathbf{X}_{t+1} \mid \mathbf{e}_{1:t+1}) \propto \mathbf{P}(\mathbf{e}_{t+1} \mid \mathbf{X}_{t+1}) \sum_{\mathbf{x}_t} \mathbf{P}(\mathbf{X}_{t+1} \mid \mathbf{x}_t) P(\mathbf{x}_t \mid \mathbf{e}_{1:t}),$$

  at a cost per step that does not grow with $t$. Prediction, smoothing and the most likely explanation are the same recursion, run forward, backward, or with the sum replaced by a max.
- The same recursion holds for continuous variables, with integrals in place of sums: exactly for linear Gaussian models, where it is the **Kalman filter**, and approximately by sampling, where it is the **particle filter**.

The end.